Telecom Customer Segmentation & Package Recommendation System

Complete Source Code

EDA • Feature Engineering • K-Means / DBSCAN Segmentation • KNN Recommender • Churn Prediction (KNN / RF / XGBoost) • SHAP Explainability

12 files

Contents

README.md

```
# Telecom Customer Segmentation & Package Recommendation System

End-to-end data-science project on the **Orange / BigML Telecom Churn** dataset
(3,333 subscribers, provided as an 80/20 split). It combines **EDA, feature
engineering, unsupervised segmentation, a KNN recommendation engine, churn
classification, and explainable AI (SHAP)** into a single reproducible pipeline.

> Why this is stronger than a standard churn project: it pairs **classification**
> with **clustering**, a **recommendation system**, telecom **domain features**,
> **explainable AI**, and a **Power BI-ready** scored dataset.

---

## Dataset

Orange Telecom Churn dataset (Kaggle / BigML). Key fields: account length,
day/evening/night/international minutes·calls·charges, voicemail & international
plan flags, customer-service calls, and the `Churn` target.

| File | Rows | Role |
|------|------|------|
| `churn-bigml-80.csv` | 2,666 | training split |
| `churn-bigml-20.csv` | 667 | holdout test split |

Overall churn rate: **14.5%** (imbalanced).

---

## Quick start

```bash
pip install -r requirements.txt
python src/run_all.py          # runs Phase 1 -> 7, ~30s
```

Run a single phase, e.g.:

```bash
python src/phase3_segmentation.py
```

All artifacts are written under `outputs/` (`figures/`, `data/`, `models/`, `reports/`).

Pipeline

Phase 1 — EDA (`phase1_eda.py`)
Churn / day-minutes / international / charges / service-call distributions,
correlation heatmap, and churned-vs-retained boxplots.

Key findings
- Heavy users (top 10% by minutes) churn **47%** vs 14.5% overall.
- Top payers (top 10% by charges) churn **63%** — high-value customers are the most at-risk.
- Customers with **≥4 service calls** churn **52%**.
- International-plan holders churn **42%** vs 11.5% without.

Phase 2 — Feature Engineering (`phase2_features.py`)
`Total Usage Minutes`, `Total Charges`, `International Usage Ratio`,
`Customer Value Score` (0–100 weighted blend of spend, tenure, engagement),
plus `Revenue Segment` (Low/Med/High) and `Usage Segment` (Light/Med/Heavy) tertiles.

Phase 3 — Customer Segmentation (`phase3_segmentation.py`)
K-Means (k chosen for business granularity; silhouette + elbow plotted) with a
DBSCAN comparison. Clusters are mapped to **5 business segments** via a greedy,
distinctiveness-based labeller:

Segment	Customers	Churn	Avg charges	Profile
High Risk Users	728	**32%**	$72.80	high-spend, high-churn → retention priority
Premium Users	836	9%	$60.10	high value, stable
Voice Heavy Users	509	10%	$60.27	heavy voicemail usage
International Users	568	13%	$44.97	high intl-usage ratio
Low Revenue Users	692	7%	$55.91	low value, low churn

Phase 4 — KNN Recommendation Engine (`phase4_recommendation.py`)
A rule-based catalogue (Basic Saver, Day Talker Pro, Evening & Night Saver,
```

```
Global Connect, Voicemail Plus, Premium Unlimited) assigns each customer a
*current* package. `NearestNeighbors` then finds each subscriber's **5 most
similar customers** (usage, charges, intl calls, account length, service calls)
and recommends the dominant neighbour package:

> *"Customers similar to you are using Package X."*

**~40% of subscribers** are flagged with a cross-sell / upsell opportunity.

### Phase 5 — Churn Prediction  (`phase5_churn.py`)
Three models — **KNN, Random Forest, XGBoost** — each in an imbalanced-learn
pipeline with **SMOTE**, tuned with **GridSearchCV** over **Stratified 5-Fold**
CV, evaluated on the holdout split.

| Model | CV ROC-AUC | Test ROC-AUC | Test F1 |
|-------|-----------|--------------|---------|
| KNN | 0.880 | 0.904 | 0.560 |
| Random Forest | 0.915 | 0.920 | 0.876 |
| **XGBoost** ■ | **0.925** | **0.924** | 0.843 |

Best model (XGBoost) on the holdout: **churn recall 0.87**, precision 0.81,
overall accuracy 0.95. Saved to `outputs/models/best_churn_model.joblib`.

### Phase 6 — Explainability  (`phase6_explainability.py`)
SHAP `TreeExplainer` on the tuned model — beeswarm summary + global importance.
Top churn drivers: **Total Charges (revenue), Voicemail plan, Voicemail
messages, International calls, Customer service calls, International plan** —
matching domain expectations.

### Phase 7 — Business Value
The operator can **identify high-value customers**, **recommend better
packages**, **reduce churn** (per-subscriber probability + risk band),
**increase ARPU**, and **target retention campaigns** at the High-Risk +
High-Value overlap.

---

## Outputs

| Path | Description |
|------|-------------|
| `outputs/figures/01..15_*.png` | All EDA, segmentation, model & SHAP plots |
| `outputs/data/telecom_features.csv` | Engineered features (Phase 2) |
| `outputs/data/telecom_segmented.csv` | + segment & package labels (Phase 3–4) |
| `outputs/data/recommendations.csv` | Per-customer package recommendations |
| `outputs/data/telecom_scored.csv` | **Power BI-ready**: features + segments + churn probability + risk band |
| `outputs/models/*.joblib` | Saved recommender & best churn model |
| `outputs/reports/*` | Insight & metric summaries |

### Power BI dashboard
Load `outputs/data/telecom_scored.csv`. Suggested visuals: churn rate by
`Segment`, `Churn Risk Band` distribution, ARPU (`Total Charges`) by segment,
`Current` vs `Recommended Package` flow, and a high-risk/high-value target list.

---

## Project structure

```
.
■■■ churn-bigml-80.csv / churn-bigml-20.csv   # raw data
■■■ requirements.txt
■■■ README.md
■■■ src/
■   ■■■ config.py   # paths, feature groups, constants
■   ■■■ utils.py   # loading, cleaning, figure helpers
■   ■■■ phase1_eda.py
■   ■■■ phase2_features.py
■   ■■■ phase3_segmentation.py
■   ■■■ phase4_recommendation.py
■   ■■■ phase5_churn.py
■   ■■■ phase6_explainability.py
■   ■■■ run_all.py   # orchestrator (Phase 1 -> 7)
■■■ outputs/   # generated: figures / data / models / reports
```
```

requirements.txt

```
pandas>=2.0
numpy>=1.24
matplotlib>=3.7
seaborn>=0.12
scikit-learn>=1.3
imbalanced-learn>=0.11
xgboost>=2.0
shap>=0.44
joblib>=1.3
```

src/config.py

```
"""
Central configuration for the Telecom Customer Segmentation &
Package Recommendation System.

All paths are resolved relative to the project root so the pipeline
runs the same way regardless of the current working directory.
"""
from __future__ import annotations

from pathlib import Path

# ---------------------------------------------------------------------------
# Paths
# ---------------------------------------------------------------------------
PROJECT_ROOT = Path(__file__).resolve().parents[1]

# Raw data (Orange / BigML telecom churn dataset, 80/20 split)
RAW_TRAIN_CSV = PROJECT_ROOT / "churn-bigml-80.csv"
RAW_TEST_CSV = PROJECT_ROOT / "churn-bigml-20.csv"

# Output folders
OUTPUTS = PROJECT_ROOT / "outputs"
FIG_DIR = OUTPUTS / "figures"
DATA_DIR = OUTPUTS / "data"
MODEL_DIR = OUTPUTS / "models"
REPORT_DIR = OUTPUTS / "reports"

for _d in (FIG_DIR, DATA_DIR, MODEL_DIR, REPORT_DIR):
    _d.mkdir(parents=True, exist_ok=True)

# Intermediate artifacts shared between phases
FEATURES_CSV = DATA_DIR / "telecom_features.csv"          # Phase 2 output
SEGMENTED_CSV = DATA_DIR / "telecom_segmented.csv"        # Phase 3 output
RECOMMENDATIONS_CSV = DATA_DIR / "recommendations.csv"    # Phase 4 output
SCORED_CSV = DATA_DIR / "telecom_scored.csv"             # Phase 5 output (Power BI ready)

# ---------------------------------------------------------------------------
# Column groups
# ---------------------------------------------------------------------------
TARGET = "Churn"

CATEGORICAL_RAW = ["International plan", "Voice mail plan"]
DROP_FOR_MODEL = ["State", "Area code"]  # high-cardinality / non-predictive id-like

# Raw usage columns
DAY_COLS = ["Total day minutes", "Total day calls", "Total day charge"]
EVE_COLS = ["Total eve minutes", "Total eve calls", "Total eve charge"]
NIGHT_COLS = ["Total night minutes", "Total night calls", "Total night charge"]
INTL_COLS = ["Total intl minutes", "Total intl calls", "Total intl charge"]

MINUTE_COLS = [
    "Total day minutes",
    "Total eve minutes",
    "Total night minutes",
    "Total intl minutes",
]
CHARGE_COLS = [
    "Total day charge",
    "Total eve charge",
    "Total night charge",
    "Total intl charge",
]

# Features used for clustering / segmentation (engineered + key raw)
SEGMENTATION_FEATURES = [
    "Total Usage Minutes",
    "Total Charges",
    "International Usage Ratio",
    "Total day minutes",
    "Total eve minutes",
    "Total night minutes",
    "Total intl minutes",
    "Number vmail messages",
    "Customer service calls",
    "Customer Value Score",
]

```

```
# Features used for the KNN recommendation engine
RECO_FEATURES = [
    "Total Usage Minutes",
    "Total Charges",
    "Total intl minutes",
    "Total intl calls",
    "Account length",
    "Customer service calls",
]

# Reproducibility
RANDOM_STATE = 42
```

src/utils.py

```
"""Shared helpers: data loading, cleaning, and figure saving."""
from __future__ import annotations

import matplotlib

matplotlib.use("Agg")  # headless backend so the pipeline runs without a display
import matplotlib.pyplot as plt
import pandas as pd

import config


def _clean(df: pd.DataFrame) -> pd.DataFrame:
    """Normalise dtypes coming from the raw CSV."""
    df = df.copy()
    # Churn / plan columns arrive as strings ("True"/"False", "Yes"/"No")
    if df[config.TARGET].dtype == object:
        df[config.TARGET] = (
            df[config.TARGET].astype(str).str.strip().str.lower().map({"true": 1, "false": 0})
        )
    else:
        df[config.TARGET] = df[config.TARGET].astype(int)

    for col in config.CATEGORICAL_RAW:
        df[col] = (
            df[col].astype(str).str.strip().str.lower().map({"yes": 1, "no": 0}).astype(int)
        )
    return df


def load_raw(split: str = "all") -> pd.DataFrame:
    """Load the raw dataset.

    split: "train" (80%), "test" (20%), or "all" (both concatenated).
    """
    train = _clean(pd.read_csv(config.RAW_TRAIN_CSV))
    test = _clean(pd.read_csv(config.RAW_TEST_CSV))
    if split == "train":
        return train.reset_index(drop=True)
    if split == "test":
        return test.reset_index(drop=True)
    return pd.concat([train, test], ignore_index=True)


def load_train_test() -> tuple[pd.DataFrame, pd.DataFrame]:
    """Return the provided 80/20 split (used as train / holdout test)."""
    return (
        _clean(pd.read_csv(config.RAW_TRAIN_CSV)).reset_index(drop=True),
        _clean(pd.read_csv(config.RAW_TEST_CSV)).reset_index(drop=True),
    )


def savefig(fig: plt.Figure, name: str) -> None:
    """Save a figure to the figures folder as PNG and close it."""
    path = config.FIG_DIR / f"{name}.png"
    fig.tight_layout()
    fig.savefig(path, dpi=130, bbox_inches="tight")
    plt.close(fig)
    print(f"  [fig] {path.relative_to(config.PROJECT_ROOT)}")


def section(title: str) -> None:
    bar = "=" * 70
    print(f"\n{bar}\n{title}\n{bar}")
```

src/phase1_eda.py

```
"""
Phase 1 - Exploratory Data Analysis.

Generates the required plots and answers the business questions:
  * Who are heavy users?
  * Who pays the most?
  * Who calls customer care frequently?
  * What drives churn?
"""
from __future__ import annotations

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

import config
from utils import load_raw, savefig, section

sns.set_theme(style="whitegrid", palette="deep")


def run() -> pd.DataFrame:
    section("PHASE 1 - EXPLORATORY DATA ANALYSIS")
    df = load_raw("all")
    print(f"Loaded {len(df):,} customers, {df.shape[1]} columns.")
    churn_rate = df[config.TARGET].mean()
    print(f"Overall churn rate: {churn_rate:.1%}")

    # --- helper engineered columns just for plotting -----------------------
    df["_total_minutes"] = df[config.MINUTE_COLS].sum(axis=1)
    df["_total_charges"] = df[config.CHARGE_COLS].sum(axis=1)
    churn_label = df[config.TARGET].map({0: "Retained", 1: "Churned"})

    # 1. Churn distribution ---------------------------------------------------
    fig, ax = plt.subplots(figsize=(6, 4))
    counts = df[config.TARGET].map({0: "Retained", 1: "Churned"}).value_counts()
    sns.barplot(x=counts.index, y=counts.values, ax=ax)
    for i, v in enumerate(counts.values):
        ax.text(i, v + 20, f"{v}\n({v/len(df):.1%})", ha="center")
    ax.set_title("Churn Distribution")
    ax.set_ylabel("Customers")
    savefig(fig, "01_churn_distribution")

    # 2. Day minutes distribution ---------------------------------------------
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.histplot(df["Total day minutes"], kde=True, bins=40, ax=ax)
    ax.axvline(df["Total day minutes"].mean(), color="red", ls="--", label="mean")
    ax.set_title("Total Day Minutes Distribution")
    ax.legend()
    savefig(fig, "02_day_minutes_distribution")

    # 3. International usage distribution -------------------------------------
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    sns.histplot(df["Total intl minutes"], kde=True, bins=30, ax=axes[0])
    axes[0].set_title("International Minutes Distribution")
    sns.countplot(x="International plan", data=df, ax=axes[1])
    axes[1].set_xticks([0, 1])
    axes[1].set_xticklabels(["No Plan", "Has Plan"])
    axes[1].set_title("International Plan Adoption")
    savefig(fig, "03_international_usage_distribution")

    # 4. Charges distribution -------------------------------------------------
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.histplot(df["_total_charges"], kde=True, bins=40, color="seagreen", ax=ax)
    ax.axvline(df["_total_charges"].mean(), color="red", ls="--", label="mean")
    ax.set_title("Total Charges Distribution")
    ax.set_xlabel("Total Charges ($)")
    ax.legend()
    savefig(fig, "04_charges_distribution")

    # 5. Customer service calls distribution ----------------------------------
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.countplot(x="Customer service calls", hue=churn_label, data=df, ax=ax)
    ax.set_title("Customer Service Calls vs Churn")
    ax.legend(title="")
    savefig(fig, "05_customer_service_calls_distribution")
```

```
    |
    |     # 6. Correlation heatmap ------------------------------------------------
    |     numeric = df.select_dtypes(include=[np.number]).drop(columns=["_total_minutes", "_total_charges"
    |     ])
    |     fig, ax = plt.subplots(figsize=(13, 10))
    |     sns.heatmap(numeric.corr(), cmap="coolwarm", center=0, annot=False, ax=ax)
    |     ax.set_title("Correlation Heatmap")
    |     savefig(fig, "06_correlation_heatmap")
    |
    |     # 7. Boxplots churned vs non-churned ----------------------------------
    |     box_features = [
    |         "Total day minutes",
    |         "_total_charges",
    |         "Customer service calls",
    |         "Total intl minutes",
    |         "Account length",
    |         "Number vmail messages",
    |     ]
    |     titles = [
    |         "Day Minutes",
    |         "Total Charges",
    |         "Customer Service Calls",
    |         "Intl Minutes",
    |         "Account Length",
    |         "Voicemail Messages",
    |     ]
    |     fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    |     for ax, feat, title in zip(axes.ravel(), box_features, titles):
    |         sns.boxplot(x=churn_label, y=df[feat], ax=ax)
    |         ax.set_title(title)
    |         ax.set_xlabel("")
    |         ax.set_ylabel("")
    |     fig.suptitle("Churned vs Retained - Feature Distributions", fontsize=14)
    |     savefig(fig, "07_boxplots_churn_vs_retained")
    |
    |     # --- answer the business questions ----------------------------------
    |     section("EDA INSIGHTS")
    |     insights = []
    |
    |     heavy_cut = df["_total_minutes"].quantile(0.9)
    |     heavy = df[df["_total_minutes"] >= heavy_cut]
    |     insights.append(
    |         f"Heavy users (top 10% by total minutes, >= {heavy_cut:.0f} min): "
    |         f"{len(heavy)} customers, avg charge ${heavy['_total_charges'].mean():.2f}, "
    |         f"churn {heavy[config.TARGET].mean():.1%} vs {churn_rate:.1%} overall."
    |     )
    |
    |     pay_cut = df["_total_charges"].quantile(0.9)
    |     top_pay = df[df["_total_charges"] >= pay_cut]
    |     insights.append(
    |         f"Top payers (top 10% by charges, >= ${pay_cut:.2f}): churn "
    |         f"{top_pay[config.TARGET].mean():.1%}; day-minutes drive most of the bill "
    |         f"(day charge corr with total charge = "
    |         f"{df['Total day charge'].corr(df['_total_charges']):.2f})."
    |     )
    |
    |     freq_care = df[df["Customer service calls"] >= 4]
    |     insights.append(
    |         f"Frequent care callers (>=4 calls): {len(freq_care)} customers, churn "
    |         f"{freq_care[config.TARGET].mean():.1%} - a major red flag vs {churn_rate:.1%} overall."
    |     )
    |
    |     intl_churn = df.groupby("International plan")[config.TARGET].mean()
    |     insights.append(
    |         "Churn drivers: international-plan holders churn "
    |         f"{intl_churn.get(1, float('nan')):.1%} vs {intl_churn.get(0, float('nan')):.1%} without; "
    |         "high day usage and >=4 service calls are the strongest churn signals."
    |     )
    |
    |     for line in insights:
    |         print(" - " + line)
    |
    |     # persist insights to a markdown report
    |     report = config.REPORT_DIR / "phase1_eda_insights.md"
    |     with open(report, "w", encoding="utf-8") as fh:
    |         fh.write("# Phase 1 - EDA Insights\n\n")
    |         fh.write(f"- Customers analysed: **{len(df):,}**\n")
    |         fh.write(f"- Overall churn rate: **{churn_rate:.1%}**\n\n")
    |         fh.write("## Business questions\n\n")
```

```
        for line in insights:
            fh.write(f"- {line}\n")
    print(f"\n[report] {report.relative_to(config.PROJECT_ROOT)}")
    return df


if __name__ == "__main__":
    run()
```

src/phase2_features.py

```
"""
Phase 2 - Feature Engineering.

Creates:
  * Total Usage Minutes
  * Total Charges
  * International Usage Ratio
  * Customer Value Score
  * Revenue Segment   (Low / Medium / High)
  * Usage Segment     (Light / Medium / Heavy)

Writes the enriched table to outputs/data/telecom_features.csv
"""
from __future__ import annotations

import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

import config
from utils import load_raw, section


def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    df["Total Usage Minutes"] = df[config.MINUTE_COLS].sum(axis=1)
    df["Total Charges"] = df[config.CHARGE_COLS].sum(axis=1)

    # share of usage that is international (avoid divide-by-zero)
    df["International Usage Ratio"] = (
        df["Total intl minutes"] / df["Total Usage Minutes"].replace(0, np.nan)
    ).fillna(0)

    df["Total Calls"] = (
        df["Total day calls"]
        + df["Total eve calls"]
        + df["Total night calls"]
        + df["Total intl calls"]
    )

    # ---- Customer Value Score ------------------------------------------
    # Weighted blend of spend, tenure and engagement, scaled to 0-100.
    scaler = MinMaxScaler()
    comps = scaler.fit_transform(
        df[["Total Charges", "Account length", "Total Calls", "Number vmail messages"]]
    )
    value = (
        0.55 * comps[:, 0]   # spend dominates value
        + 0.20 * comps[:, 1]  # tenure / loyalty
        + 0.20 * comps[:, 2]  # call engagement
        + 0.05 * comps[:, 3]  # voicemail engagement
    )
    df["Customer Value Score"] = (value * 100).round(2)

    # ---- Categorical segments (tertiles) -------------------------------
    df["Revenue Segment"] = pd.qcut(
        df["Total Charges"], q=3, labels=["Low", "Medium", "High"]
    )
    df["Usage Segment"] = pd.qcut(
        df["Total Usage Minutes"], q=3, labels=["Light", "Medium", "Heavy"]
    )

    return df


def run() -> pd.DataFrame:
    section("PHASE 2 - FEATURE ENGINEERING")
    df = load_raw("all")
    df = add_features(df)

    new_cols = [
        "Total Usage Minutes",
        "Total Charges",
        "International Usage Ratio",
        "Customer Value Score",
        "Revenue Segment",
```

```
        "Usage Segment",
    ]
    print("Engineered features:")
    print(df[new_cols].describe(include="all").T.to_string())

    df.to_csv(config.FEATURES_CSV, index=False)
    print(f"\n[data] {config.FEATURES_CSV.relative_to(config.PROJECT_ROOT)} "
          f"({df.shape[0]} rows x {df.shape[1]} cols)")
    return df


if __name__ == "__main__":
    run()
```

src/phase3_segmentation.py

```
"""
Phase 3 - Customer Segmentation (K-Means + DBSCAN).

Clusters customers on usage / value features, picks k by silhouette,
profiles each cluster and maps it to a business segment name:
  Premium Users, Voice Heavy Users, International Users,
  Low Revenue Users, High Risk Users.

Writes outputs/data/telecom_segmented.csv
"""
from __future__ import annotations

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.cluster import DBSCAN, KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

import config
from phase2_features import add_features
from utils import load_raw, savefig, section

sns.set_theme(style="whitegrid")


def _load_features() -> pd.DataFrame:
    if config.FEATURES_CSV.exists():
        return pd.read_csv(config.FEATURES_CSV)
    return add_features(load_raw("all"))


def _choose_k(X: np.ndarray, k_range=range(2, 9)) -> tuple[int, list, list]:
    inertias, sils = [], []
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=config.RANDOM_STATE, n_init=10)
        labels = km.fit_predict(X)
        inertias.append(km.inertia_)
        sils.append(silhouette_score(X, labels))
    best_k = list(k_range)[int(np.argmax(sils))]
    return best_k, inertias, sils


# Number of clusters for the business segmentation. The data is fairly
# homogeneous so the silhouette-optimal k is small (2); we deliberately
# segment into the 5 business-relevant groups the project requires.
SEGMENT_K = 5


def _label_clusters(cluster_profiles: pd.DataFrame) -> dict:
    """Greedily assign each cluster a unique business segment name based on
    which dimension it is most extreme in relative to the other clusters."""
    remaining = set(cluster_profiles.index)
    mapping: dict = {}

    # composite risk: actual churn dominates, service-call intensity is a
    # light tie-breaker (all clusters have similar service-call means).
    sc = cluster_profiles["Customer service calls"]
    sc_norm = (sc - sc.min()) / (sc.max() - sc.min() + 1e-9)
    risk = cluster_profiles["churn_rate"] + 0.1 * sc_norm

    # (label, ranking series) in priority order
    priorities = [
        ("High Risk Users", risk),
        ("International Users", cluster_profiles["International Usage Ratio"]),
        ("Voice Heavy Users", cluster_profiles["Number vmail messages"]),
        ("Premium Users", cluster_profiles["Customer Value Score"]),
    ]
    for label, series in priorities:
        cand = series[list(remaining)].sort_values(ascending=False)
        if len(cand):
            chosen = cand.index[0]
            mapping[chosen] = label
            remaining.discard(chosen)

```

```
    # whatever is left (lowest value) -> Low Revenue Users
    for cl in sorted(remaining, key=lambda c: cluster_profiles.loc[c, "Customer Value Score"]):
        mapping[cl] = "Low Revenue Users"
    return mapping


def run() -> pd.DataFrame:
    section("PHASE 3 - CUSTOMER SEGMENTATION")
    df = _load_features()

    feats = [c for c in config.SEGMENTATION_FEATURES if c in df.columns]
    scaler = StandardScaler()
    X = scaler.fit_transform(df[feats])

    # ---- choose k ------------------------------------------------------
    k_range = range(2, 9)
    best_k, inertias, sils = _choose_k(X, k_range)
    seg_k = SEGMENT_K
    print(f"Silhouette-optimal k = {best_k} (silhouette={max(sils):.3f}); "
          f"segmenting into k={seg_k} for business granularity.")

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].plot(list(k_range), inertias, "o-")
    axes[0].axvline(seg_k, color="red", ls="--", label=f"chosen k={seg_k}")
    axes[0].set(title="Elbow (Inertia)", xlabel="k", ylabel="Inertia")
    axes[0].legend()
    axes[1].plot(list(k_range), sils, "o-", color="darkorange")
    axes[1].axvline(best_k, color="green", ls="--", label=f"best silhouette k={best_k}")
    axes[1].axvline(seg_k, color="red", ls="--", label=f"chosen k={seg_k}")
    axes[1].set(title="Silhouette Score", xlabel="k", ylabel="silhouette")
    axes[1].legend()
    savefig(fig, "08_kmeans_k_selection")

    # ---- K-Means -------------------------------------------------------
    km = KMeans(n_clusters=seg_k, random_state=config.RANDOM_STATE, n_init=10)
    df["cluster"] = km.fit_predict(X)

    # ---- profile each cluster ------------------------------------------
    df["churn_rate"] = df[config.TARGET]
    prof = df.groupby("cluster").agg(
        size=("cluster", "size"),
        churn_rate=(config.TARGET, "mean"),
        avg_charges=("Total Charges", "mean"),
        avg_minutes=("Total Usage Minutes", "mean"),
        intl_ratio=("International Usage Ratio", "mean"),
        vmail=("Number vmail messages", "mean"),
        service_calls=("Customer service calls", "mean"),
        value_score=("Customer Value Score", "mean"),
    )
    print("\nCluster profiles:")
    print(prof.round(2).to_string())

    # map cluster -> unique business label
    cluster_profiles = df.groupby("cluster").agg(
        {
            "Customer service calls": "mean",
            "International Usage Ratio": "mean",
            "Number vmail messages": "mean",
            "Customer Value Score": "mean",
            config.TARGET: "mean",
        }
    ).rename(columns={config.TARGET: "churn_rate"})

    mapping = _label_clusters(cluster_profiles)
    df["Segment"] = df["cluster"].map(mapping)
    print("\nCluster -> Segment mapping:")
    for cl, name in mapping.items():
        print(f"  cluster {cl} -> {name}")

    seg_summary = df.groupby("Segment").agg(
        customers=("Segment", "size"),
        churn_rate=(config.TARGET, "mean"),
        avg_charges=("Total Charges", "mean"),
        avg_value=("Customer Value Score", "mean"),
    ).round(2)
    print("\nSegment summary:")
    print(seg_summary.to_string())

    # ---- PCA scatter of segments ---------------------------------------
```

```
    pca = PCA(n_components=2, random_state=config.RANDOM_STATE)
    coords = pca.fit_transform(X)
    df["_pc1"], df["_pc2"] = coords[:, 0], coords[:, 1]
    fig, ax = plt.subplots(figsize=(9, 6))
    sns.scatterplot(x="_pc1", y="_pc2", hue="Segment", data=df, s=18, alpha=0.7, ax=ax)
    ax.set_title(f"Customer Segments (K-Means, k={best_k}) - PCA projection")
    ax.set(xlabel="PC1", ylabel="PC2")
    ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
    savefig(fig, "09_segments_pca_scatter")

    # segment size bar
    fig, ax = plt.subplots(figsize=(8, 4))
    order = df["Segment"].value_counts().index
    sns.countplot(y="Segment", data=df, order=order, ax=ax)
    ax.set_title("Segment Sizes")
    savefig(fig, "10_segment_sizes")

    # ---- DBSCAN comparison ------------------------------------------------
    db = DBSCAN(eps=2.0, min_samples=10)
    db_labels = db.fit_predict(X)
    n_clusters = len(set(db_labels)) - (1 if -1 in db_labels else 0)
    n_noise = int((db_labels == -1).sum())
    df["dbscan_cluster"] = db_labels
    print(f"\nDBSCAN (eps=2.0, min_samples=10): {n_clusters} clusters, "
          f"{n_noise} noise points ({n_noise/len(df):.1%}).")
    if n_clusters >= 2:
        mask = db_labels != -1
        print(f"  DBSCAN silhouette (excl. noise): "
              f"{silhouette_score(X[mask], db_labels[mask]):.3f}")

    fig, ax = plt.subplots(figsize=(9, 6))
    sns.scatterplot(
        x=df["_pc1"], y=df["_pc2"],
        hue=df["dbscan_cluster"].astype(str), s=18, alpha=0.7, ax=ax, legend="brief"
    )
    ax.set_title("DBSCAN Clusters (-1 = noise) - PCA projection")
    ax.set(xlabel="PC1", ylabel="PC2")
    ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8, title="cluster")
    savefig(fig, "11_dbscan_pca_scatter")

    # ---- persist ----------------------------------------------------------
    df = df.drop(columns=["churn_rate", "_pc1", "_pc2"])
    df.to_csv(config.SEGMENTED_CSV, index=False)
    print(f"\n[data] {config.SEGMENTED_CSV.relative_to(config.PROJECT_ROOT)}")

    seg_summary.to_csv(config.REPORT_DIR / "phase3_segment_summary.csv")
    return df


if __name__ == "__main__":
    run()
```

src/phase4_recommendation.py

```
"""
Phase 4 - KNN Package Recommendation Engine.

For every subscriber we find the 5 most similar customers (by usage,
charges, international calls, account length and service calls) and
recommend the package most common among those neighbours:

    "Customers similar to you are using Package X."

A rule-based package catalogue assigns each customer a current package;
the recommender then surfaces the dominant neighbour package, flagging
cross-sell / upsell opportunities where it differs.

Writes outputs/data/recommendations.csv
"""
from __future__ import annotations

import joblib
import numpy as np
import pandas as pd
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

import config
from phase3_segmentation import run as run_phase3
from utils import section

PACKAGES = [
    "Basic Saver",
    "Day Talker Pro",
    "Evening & Night Saver",
    "Global Connect",
    "Voicemail Plus",
    "Premium Unlimited",
]


def assign_package(row: pd.Series, q: dict) -> str:
    """Rule-based 'current package' from a customer's usage profile."""
    if row["Customer Value Score"] >= q["value_high"] and row["Total Charges"] >= q["charge_high"]:
        return "Premium Unlimited"
    if row["International Usage Ratio"] >= q["intl_high"] or row["Total intl minutes"] >= q["intl_mi
    n_high"]:
        return "Global Connect"
    if row["Number vmail messages"] >= q["vmail_high"]:
        return "Voicemail Plus"
    if row["Total day minutes"] >= q["day_high"]:
        return "Day Talker Pro"
    if (row["Total eve minutes"] + row["Total night minutes"]) >= q["evenight_high"]:
        return "Evening & Night Saver"
    return "Basic Saver"


def build_package_catalog(df: pd.DataFrame) -> pd.Series:
    q = {
        "value_high": df["Customer Value Score"].quantile(0.75),
        "charge_high": df["Total Charges"].quantile(0.70),
        "intl_high": df["International Usage Ratio"].quantile(0.85),
        "intl_min_high": df["Total intl minutes"].quantile(0.85),
        "vmail_high": 20,
        "day_high": df["Total day minutes"].quantile(0.75),
        "evenight_high": (df["Total eve minutes"] + df["Total night minutes"]).quantile(0.75),
    }
    return df.apply(lambda r: assign_package(r, q), axis=1)


def run() -> pd.DataFrame:
    section("PHASE 4 - KNN PACKAGE RECOMMENDATION ENGINE")

    if config.SEGMENTED_CSV.exists():
        df = pd.read_csv(config.SEGMENTED_CSV)
    else:
        df = run_phase3()

    # current package per customer
    df["Current Package"] = build_package_catalog(df)
    print("Current package distribution:")
```

```
    print(df["Current Package"].value_counts().to_string())

    # ---- fit nearest-neighbours model ---------------------------------
    feats = [c for c in config.RECO_FEATURES if c in df.columns]
    scaler = StandardScaler()
    X = scaler.fit_transform(df[feats])

    k = 6  # 5 neighbours + self
    nn = NearestNeighbors(n_neighbors=k, algorithm="auto")
    nn.fit(X)
    _, idx = nn.kneighbors(X)

    pkgs = df["Current Package"].to_numpy()
    recommended, n_diff_pkg, examples = [], [], []
    for i in range(len(df)):
        neigh = idx[i][1:]  # drop self
        neigh_pkgs = pkgs[neigh]
        vals, counts = np.unique(neigh_pkgs, return_counts=True)
        rec = vals[int(np.argmax(counts))]
        recommended.append(rec)

    df["Recommended Package"] = recommended
    df["Upsell Opportunity"] = df["Recommended Package"] != df["Current Package"]
    df["Recommendation Text"] = df["Recommended Package"].map(
        lambda p: f"Customers similar to you are using {p}."
    )

    n_up = int(df["Upsell Opportunity"].sum())
    print(f"\nNeighbours used per customer: 5")
    print(f"Cross-sell / upsell opportunities flagged: {n_up:,} ({n_up/len(df):.1%})")
    print("\nRecommended package distribution:")
    print(df["Recommended Package"].value_counts().to_string())

    # show a few example recommendations
    print("\nSample recommendations:")
    cols = ["Total Charges", "Total intl minutes", "Current Package",
            "Recommended Package", "Recommendation Text"]
    print(df[cols].head(8).to_string(index=False))

    # ---- persist ------------------------------------------------------
    out_cols = [
        "Account length", "Total Usage Minutes", "Total Charges",
        "Total intl minutes", "Customer service calls", "Customer Value Score",
        "Segment", "Current Package", "Recommended Package",
        "Upsell Opportunity", "Recommendation Text",
    ]
    out_cols = [c for c in out_cols if c in df.columns]
    df[out_cols].to_csv(config.RECOMMENDATIONS_CSV, index=False)
    print(f"\n[data] {config.RECOMMENDATIONS_CSV.relative_to(config.PROJECT_ROOT)}")

    joblib.dump(
        {"model": nn, "scaler": scaler, "features": feats, "packages": pkgs},
        config.MODEL_DIR / "knn_recommender.joblib",
    )
    # carry package columns forward
    df.to_csv(config.SEGMENTED_CSV, index=False)
    return df


if __name__ == "__main__":
    run()
```

src/phase5_churn.py

```
"""
Phase 5 - Churn Prediction.

Three models (KNN, Random Forest, XGBoost), each in an imbalanced-learn
pipeline with SMOTE, tuned with GridSearchCV over Stratified K-Fold.
The provided 80/20 split is used as train / holdout test.

Outputs metrics, ROC curves, confusion matrix, the best model, and a
Power BI-ready scored table (out-of-fold churn probabilities for all
customers) at outputs/data/telecom_scored.csv
"""
from __future__ import annotations

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    classification_report,
    f1_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_predict
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier

import config
from phase2_features import add_features
from utils import load_train_test, savefig, section

sns.set_theme(style="whitegrid")

MODEL_FEATURES = [
    "Account length", "International plan", "Voice mail plan",
    "Number vmail messages",
    "Total day minutes", "Total day calls", "Total day charge",
    "Total eve minutes", "Total eve calls", "Total eve charge",
    "Total night minutes", "Total night calls", "Total night charge",
    "Total intl minutes", "Total intl calls", "Total intl charge",
    "Customer service calls",
    "Total Usage Minutes", "Total Charges", "International Usage Ratio",
]


def _models() -> dict:
    rs = config.RANDOM_STATE
    return {
        "KNN": (
            ImbPipeline([
                ("scaler", StandardScaler()),
                ("smote", SMOTE(random_state=rs)),
                ("clf", KNeighborsClassifier()),
            ]),
            {"clf__n_neighbors": [5, 11, 21], "clf__weights": ["uniform", "distance"]},
        ),
        "RandomForest": (
            ImbPipeline([
                ("smote", SMOTE(random_state=rs)),
                ("clf", RandomForestClassifier(random_state=rs, n_jobs=-1)),
            ]),
            {"clf__n_estimators": [300], "clf__max_depth": [None, 12],
             "clf__min_samples_leaf": [1, 3]},
        ),
        "XGBoost": (
            ImbPipeline([
                ("smote", SMOTE(random_state=rs)),
                ("clf", XGBClassifier(
                    random_state=rs, eval_metric="logloss",
                    tree_method="hist", n_jobs=-1)),
            ]),
```

```
            {"clf__n_estimators": [300], "clf__max_depth": [4, 6],
             "clf__learning_rate": [0.05, 0.1]},
        ),
    }


def run() -> dict:
    section("PHASE 5 - CHURN PREDICTION")
    train, test = load_train_test()
    train, test = add_features(train), add_features(test)

    X_tr, y_tr = train[MODEL_FEATURES], train[config.TARGET]
    X_te, y_te = test[MODEL_FEATURES], test[config.TARGET]
    print(f"Train: {len(X_tr):,} (churn {y_tr.mean():.1%})  |  "
          f"Test: {len(X_te):,} (churn {y_te.mean():.1%})")

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=config.RANDOM_STATE)
    results, fitted = {}, {}

    for name, (pipe, grid) in _models().items():
        print(f"\n>>> Tuning {name} ...")
        gs = GridSearchCV(pipe, grid, scoring="roc_auc", cv=cv, n_jobs=-1)
        gs.fit(X_tr, y_tr)
        best = gs.best_estimator_
        proba = best.predict_proba(X_te)[:, 1]
        pred = best.predict(X_te)
        results[name] = {
            "cv_roc_auc": gs.best_score_,
            "test_roc_auc": roc_auc_score(y_te, proba),
            "test_f1": f1_score(y_te, pred),
            "best_params": gs.best_params_,
            "proba": proba,
            "pred": pred,
        }
        fitted[name] = best
        print(f"    best params: {gs.best_params_}")
        print(f"    CV ROC-AUC={gs.best_score_:.3f} | "
              f"Test ROC-AUC={results[name]['test_roc_auc']:.3f} | "
              f"Test F1={results[name]['test_f1']:.3f}")

    # ---- comparison table ----------------------------------------------
    section("MODEL COMPARISON (holdout test)")
    comp = pd.DataFrame({
        n: {"CV ROC-AUC": r["cv_roc_auc"],
            "Test ROC-AUC": r["test_roc_auc"],
            "Test F1": r["test_f1"]}
        for n, r in results.items()
    }).T.round(4)
    print(comp.to_string())
    comp.to_csv(config.REPORT_DIR / "phase5_model_comparison.csv")

    best_name = comp["Test ROC-AUC"].idxmax()
    best_model = fitted[best_name]
    print(f"\nBest model: {best_name}")
    print("\nClassification report (best model on holdout):")
    print(classification_report(y_te, results[best_name]["pred"],
                                target_names=["Retained", "Churned"]))

    # ---- ROC curves ----------------------------------------------------
    fig, ax = plt.subplots(figsize=(7, 6))
    for name, r in results.items():
        fpr, tpr, _ = roc_curve(y_te, r["proba"])
        ax.plot(fpr, tpr, label=f"{name} (AUC={r['test_roc_auc']:.3f})")
    ax.plot([0, 1], [0, 1], "k--", alpha=0.4)
    ax.set(title="ROC Curves - Churn Models", xlabel="False Positive Rate",
           ylabel="True Positive Rate")
    ax.legend()
    savefig(fig, "12_roc_curves")

    # ---- confusion matrix (best) ---------------------------------------
    fig, ax = plt.subplots(figsize=(5, 4))
    ConfusionMatrixDisplay.from_predictions(
        y_te, results[best_name]["pred"],
        display_labels=["Retained", "Churned"], cmap="Blues", ax=ax)
    ax.set_title(f"Confusion Matrix - {best_name}")
    savefig(fig, "13_confusion_matrix")

    # ---- persist best model --------------------------------------------
    joblib.dump(
```

```
        {"model": best_model, "features": MODEL_FEATURES, "name": best_name},
        config.MODEL_DIR / "best_churn_model.joblib",
    )
    print(f"[model] {(config.MODEL_DIR / 'best_churn_model.joblib').relative_to(config.PROJECT_ROOT)
    }")

    # ---- Power BI-ready scored table (out-of-fold probs for everyone) ----
    tr, te = load_train_test()
    full = add_features(pd.concat([tr, te], ignore_index=True))
    Xf, yf = full[MODEL_FEATURES], full[config.TARGET]
    oof = cross_val_predict(best_model, Xf, yf, cv=cv, method="predict_proba", n_jobs=-1)[:, 1]
    full["Churn Probability"] = oof.round(4)
    full["Churn Risk Band"] = pd.cut(
        full["Churn Probability"], bins=[-0.01, 0.3, 0.6, 1.0],
        labels=["Low", "Medium", "High"])

    # merge segment / package info if available
    if config.SEGMENTED_CSV.exists():
        seg = pd.read_csv(config.SEGMENTED_CSV)
        for col in ["Segment", "Current Package", "Recommended Package",
                    "Revenue Segment", "Usage Segment"]:
            if col in seg.columns and len(seg) == len(full):
                full[col] = seg[col].values

    full.to_csv(config.SCORED_CSV, index=False)
    print(f"[data] {config.SCORED_CSV.relative_to(config.PROJECT_ROOT)} "
          f"(Power BI ready, {full.shape[0]} rows)")

    return {"results": results, "best_name": best_name, "comparison": comp}


if __name__ == "__main__":
    run()
```

src/phase6_explainability.py

```
"""
Phase 6 - Explainability (SHAP).

Explains the tuned tree model's churn predictions. Produces a SHAP
summary (beeswarm) plot and a global feature-importance bar plot, and
prints the top churn drivers - expected to include Customer service
calls, International plan, Day minutes/charges and total revenue.
"""
from __future__ import annotations

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import shap
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

import config
from phase2_features import add_features
from phase5_churn import MODEL_FEATURES, run as run_phase5
from utils import load_train_test, savefig, section


def _get_tree_step(pipeline):
    """Extract the underlying tree classifier from the imblearn pipeline."""
    clf = pipeline.named_steps["clf"]
    return clf


def run() -> None:
    section("PHASE 6 - EXPLAINABILITY (SHAP)")

    bundle_path = config.MODEL_DIR / "best_churn_model.joblib"
    if not bundle_path.exists():
        run_phase5()
    bundle = joblib.load(bundle_path)
    pipeline, name = bundle["model"], bundle["name"]
    clf = _get_tree_step(pipeline)

    # SHAP TreeExplainer needs a tree model. If the best model is KNN,
    # fall back to fitting an XGBoost on the same data for explanation.
    train, test = load_train_test()
    train, test = add_features(train), add_features(test)
    X_tr = train[MODEL_FEATURES]
    X_te = test[MODEL_FEATURES]

    if not isinstance(clf, (RandomForestClassifier, XGBClassifier)):
        print(f"Best model ({name}) is not tree-based; fitting XGBoost for SHAP.")
        clf = XGBClassifier(
            random_state=config.RANDOM_STATE, eval_metric="logloss",
            tree_method="hist", n_jobs=-1, n_estimators=300, max_depth=6,
            learning_rate=0.1)
        clf.fit(X_tr, train[config.TARGET])
        name = "XGBoost (for SHAP)"

    print(f"Explaining model: {name}")
    explainer = shap.TreeExplainer(clf)
    shap_values = explainer.shap_values(X_te)

    # RandomForest returns a list [class0, class1]; pick churn class
    if isinstance(shap_values, list):
        shap_values = shap_values[1]
    # newer SHAP may return (n, features, classes)
    if shap_values.ndim == 3:
        shap_values = shap_values[:, :, 1]

    # ---- beeswarm summary ------------------------------------------------
    plt.figure()
    shap.summary_plot(shap_values, X_te, show=False, max_display=15)
    fig = plt.gcf()
    fig.suptitle("SHAP Summary - Churn Drivers", y=1.02)
    savefig(fig, "14_shap_summary_beeswarm")

    # ---- global importance bar -------------------------------------------
    plt.figure()
    shap.summary_plot(shap_values, X_te, plot_type="bar", show=False, max_display=15)
```

```
    fig = plt.gcf()
    fig.suptitle("SHAP Global Feature Importance", y=1.02)
    savefig(fig, "15_shap_importance_bar")

    # ---- ranked drivers ----------------------------------------------------
    importance = (
        pd.Series(np.abs(shap_values).mean(axis=0), index=MODEL_FEATURES)
        .sort_values(ascending=False)
    )
    print("\nTop churn drivers (mean |SHAP|):")
    print(importance.head(10).round(4).to_string())
    importance.to_csv(config.REPORT_DIR / "phase6_shap_importance.csv",
                      header=["mean_abs_shap"])
    print(f"\n[report] {(config.REPORT_DIR / 'phase6_shap_importance.csv').relative_to(config.PROJECT_ROOT)}")


if __name__ == "__main__":
    run()
```

src/run_all.py

```
"""
Run the full Telecom Customer Segmentation & Package Recommendation
pipeline, Phase 1 -> Phase 6, then print the Phase 7 business summary.

Usage:
    python src/run_all.py
"""
from __future__ import annotations

import time

import phase1_eda
import phase2_features
import phase3_segmentation
import phase4_recommendation
import phase5_churn
import phase6_explainability
from utils import section


def main() -> None:
    t0 = time.time()
    phase1_eda.run()
    phase2_features.run()
    phase3_segmentation.run()
    phase4_recommendation.run()
    phase5_churn.run()
    phase6_explainability.run()

    section("PHASE 7 - BUSINESS VALUE")
    print(
        "The pipeline lets a telecom operator:\n"
        "  * Identify high-value customers via the Customer Value Score & segments.\n"
        "  * Recommend better packages with the KNN engine (cross-sell / upsell flags).\n"
        "  * Reduce churn by scoring every subscriber's churn probability & risk band.\n"
        "  * Increase ARPU by moving Low-Revenue users toward fitting paid packages.\n"
        "  * Improve retention campaigns by targeting High-Risk + High-Value customers.\n\n"
        "All artifacts are in outputs/ (figures, models, reports, data).\n"
        "outputs/data/telecom_scored.csv is ready to load into Power BI."
    )
    print(f"\nDone in {time.time() - t0:.1f}s.")


if __name__ == "__main__":
    main()
```

make_code_pdf.py

```
"""Generate a PDF containing the full project source code."""
from pathlib import Path

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.platypus import (
    PageBreak,
    Paragraph,
    Preformatted,
    SimpleDocTemplate,
    Spacer,
)

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "Telecom_Segmentation_Code.pdf"

# Files to include, in logical reading order
FILES = [
    "README.md",
    "requirements.txt",
    "src/config.py",
    "src/utils.py",
    "src/phase1_eda.py",
    "src/phase2_features.py",
    "src/phase3_segmentation.py",
    "src/phase4_recommendation.py",
    "src/phase5_churn.py",
    "src/phase6_explainability.py",
    "src/run_all.py",
    "make_code_pdf.py",
]

WRAP = 100  # soft-wrap long lines so nothing overflows the page


def softwrap(line: str, width: int = WRAP) -> list[str]:
    if len(line) <= width:
        return [line]
    out, cur = [], line
    while len(cur) > width:
        out.append(cur[:width])
        cur = "    " + cur[width:]  # indent continuation
    out.append(cur)
    return out


def main() -> None:
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        "title", parent=styles["Title"], fontSize=22, leading=26)
    sub_style = ParagraphStyle(
        "sub", parent=styles["Normal"], alignment=TA_CENTER, fontSize=12,
        textColor=colors.HexColor("#555555"), spaceBefore=6)
    file_style = ParagraphStyle(
        "file", parent=styles["Heading2"], fontSize=13,
        textColor=colors.HexColor("#1a4d7a"), spaceBefore=4, spaceAfter=6)
    code_style = ParagraphStyle(
        "code", parent=styles["Code"], fontName="Courier", fontSize=7.2,
        leading=8.6, textColor=colors.HexColor("#1a1a1a"))
    toc_style = ParagraphStyle(
        "toc", parent=styles["Normal"], fontName="Courier", fontSize=9,
        leading=14)

    doc = SimpleDocTemplate(
        str(OUT), pagesize=letter,
        leftMargin=0.6 * inch, rightMargin=0.6 * inch,
        topMargin=0.7 * inch, bottomMargin=0.6 * inch,
        title="Telecom Customer Segmentation - Source Code",
        author="triaz.malik")

    story = []

    # ---- title page ----
    story.append(Spacer(1, 2.2 * inch))
```

```
    story.append(Paragraph(
        "Telecom Customer Segmentation<br/>&amp; Package Recommendation System",
        title_style))
    story.append(Paragraph("Complete Source Code", sub_style))
    story.append(Paragraph(
        "EDA &bull; Feature Engineering &bull; K-Means / DBSCAN Segmentation "
        "&bull; KNN Recommender &bull; Churn Prediction (KNN / RF / XGBoost) "
        "&bull; SHAP Explainability", sub_style))
    story.append(Spacer(1, 0.5 * inch))
    story.append(Paragraph(f"{len(FILES)} files", sub_style))

    # ---- table of contents ----
    story.append(PageBreak())
    story.append(Paragraph("Contents", file_style))
    story.append(Spacer(1, 6))
    for i, rel in enumerate(FILES, 1):
        story.append(Paragraph(f"{i:>2}.  {rel}", toc_style))

    # ---- each file ----
    for rel in FILES:
        path = ROOT / rel
        story.append(PageBreak())
        story.append(Paragraph(rel, file_style))
        try:
            text = path.read_text(encoding="utf-8")
        except Exception as exc:  # noqa: BLE001
            story.append(Paragraph(f"[could not read: {exc}]", styles["Normal"]))
            continue
        numbered = []
        for n, raw in enumerate(text.splitlines(), 1):
            for j, piece in enumerate(softwrap(raw.rstrip("\n"))):
                prefix = f"{n:>4} | " if j == 0 else "     | "
                numbered.append(prefix + piece)
        story.append(Preformatted("\n".join(numbered), code_style))

    doc.build(story)
    print(f"Wrote {OUT} ({OUT.stat().st_size/1024:.0f} KB)")


if __name__ == "__main__":
    main()
```